

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Fila.h"
4
5  void inicializarFila(ListaFila* fila) {
6      fila->primeiro = NULL;
7      fila->ultimo = NULL;
8  }
9
10 int estaVazia(ListaFila* fila) {
11     return (fila->primeiro == NULL);
12 }
13
14 void inserirElemento(ListaFila* fila, int numero) {
15     CelulaFila* novoNo = (CelulaFila*)malloc(sizeof(CelulaFila));
16     if (novoNo == NULL) return;
17     novoNo->dado = numero;
18     novoNo->proximo = NULL;
19     if (estaVazia(fila)) {
20         fila->primeiro = novoNo;
21     } else {
22         fila->ultimo->proximo = novoNo;
23     }
24     fila->ultimo = novoNo;
25 }
26
27 int retirarElemento(ListaFila* fila) {
28     if (estaVazia(fila)) return -1;
29     CelulaFila* temporario = fila->primeiro;
30     int conteudo = temporario->dado;
31     fila->primeiro = temporario->proximo;
32     if (fila->primeiro == NULL) {
33         fila->ultimo = NULL;
34     }
35     free(temporario);
36     return conteudo;
37 }
38
39 void mostrarFila(ListaFila* fila) {
40     if (estaVazia(fila)) {
41         printf("[Fila Vazia]\n");
42         return;
43     }
44     ListaFila aux;
45     inicializarFila(&aux);
46     while (!estaVazia(fila)) {
47         int v = retirarElemento(fila);
48         printf("%d ", v);
49         inserirElemento(&aux, v);
50     }
51     printf("\n");
52     while (!estaVazia(&aux)) {
53         inserirElemento(fila, retirarElemento(&aux));
54     }
55 }
56
57 void destruirFila(ListaFila* fila) {
58     while (!estaVazia(fila)) {
59         retirarElemento(fila);
60     }
61 }
62
63 int buscarElementoFila(ListaFila* fila, int numero) {
64     ListaFila aux;
65     inicializarFila(&aux);
66     int posicao = 1;

```

```

67     int encontrado = -1;
68     while (!estaVazia(fila)) {
69         int v = retirarElemento(fila);
70         if (v == numero && encontrado == -1) {
71             encontrado = posicao;
72         }
73         inserirElemento(&aux, v);
74         posicao++;
75     }
76     while (!estaVazia(&aux)) {
77         inserirElemento(fila, retirarElemento(&aux));
78     }
79     return encontrado;
80 }
81
82 void buscarEEditarFila(ListaFila* fila, int numeroAntigo, int numeroNovo) {
83     ListaFila aux;
84     inicializarFila(&aux);
85     while (!estaVazia(fila)) {
86         int v = retirarElemento(fila);
87         if (v == numeroAntigo) {
88             v = numeroNovo;
89         }
90         inserirElemento(&aux, v);
91     }
92     while (!estaVazia(&aux)) {
93         inserirElemento(fila, retirarElemento(&aux));
94     }
95 }
96
97 int buscarERemoverFila(ListaFila* fila, int numero) {
98     ListaFila aux;
99     inicializarFila(&aux);
100    int encontrado = 0;
101    while (!estaVazia(fila)) {
102        int v = retirarElemento(fila);
103        if (v == numero && !encontrado) {
104            encontrado = 1;
105        } else {
106            inserirElemento(&aux, v);
107        }
108    }
109    while (!estaVazia(&aux)) {
110        inserirElemento(fila, retirarElemento(&aux));
111    }
112    return encontrado;
113 }
114
115 void eliminarDuplicados(ListaFila* fila) {
116     ListaFila auxLimpa;
117     inicializarFila(&auxLimpa);
118     while (!estaVazia(fila)) {
119         int atual = retirarElemento(fila);
120         ListaFila tempVerificadora;
121         inicializarFila(&tempVerificadora);
122         int duplicado = 0;
123         while (!estaVazia(&auxLimpa)) {
124             int vVerifica = retirarElemento(&auxLimpa);
125             if (vVerifica == atual) {
126                 duplicado = 1;
127             }
128             inserirElemento(&tempVerificadora, vVerifica);
129         }
130         while (!estaVazia(&tempVerificadora)) {
131             inserirElemento(&auxLimpa, retirarElemento(&tempVerificadora));
132         }

```

```
133         if (!duplicado) {
134             inserirElemento(&auxLimpa, atual);
135         }
136     }
137     while (!estaVazia(&auxLimpa)) {
138         inserirElemento(fila, retirarElemento(&auxLimpa));
139     }
140 }
141
142 void removerNumerosPares(ListaFila* fila) {
143     ListaFila aux;
144     inicializarFila(&aux);
145     while (!estaVazia(fila)) {
146         int v = retirarElemento(fila);
147         if (v % 2 != 0) {
148             inserirElemento(&aux, v);
149         }
150     }
151     while (!estaVazia(&aux)) {
152         inserirElemento(fila, retirarElemento(&aux));
153     }
154 }
```

```
1  #ifndef FILA_H
2  #define FILA_H
3
4  typedef struct CelulaFila {
5      int dado;
6      struct CelulaFila* proximo;
7  } CelulaFila;
8
9  typedef struct {
10     CelulaFila* primeiro;
11     CelulaFila* ultimo;
12 } ListaFila;
13
14 void inicializarFila(ListaFila* fila);
15 int estaVazia(ListaFila* fila);
16 void inserirElemento(ListaFila* fila, int numero);
17 int retirarElemento(ListaFila* fila);
18 void mostrarFila(ListaFila* fila);
19 void destruirFila(ListaFila* fila);
20
21 int buscarElementoFila(ListaFila* fila, int numero);
22 void buscarEEditarFila(ListaFila* fila, int numeroAntigo, int numeroNovo);
23 int buscarERemoverFila(ListaFila* fila, int numero);
24 void eliminarDuplicados(ListaFila* fila);
25 void removerNumerosPares(ListaFila* fila);
26
27 #endif
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Lista.h"
4
5  Celula* criarLista() {
6      return NULL;
7  }
8
9  int listaVazia(Celula* sequencia) {
10     return (sequencia == NULL);
11 }
12
13 void inserirInicio(Celula** sequencia, int numero) {
14     Celula* novoNo = (Celula*)malloc(sizeof(Celula));
15     if (novoNo == NULL) return;
16     novoNo->dado = numero;
17     novoNo->proximo = *sequencia;
18     *sequencia = novoNo;
19 }
20
21 void inserirFim(Celula** sequencia, int numero) {
22     Celula* novoNo = (Celula*)malloc(sizeof(Celula));
23     if (novoNo == NULL) return;
24     novoNo->dado = numero;
25     novoNo->proximo = NULL;
26     if (listaVazia(*sequencia)) {
27         *sequencia = novoNo;
28         return;
29     }
30     Celula* auxiliar = *sequencia;
31     while (auxiliar->proximo != NULL) {
32         auxiliar = auxiliar->proximo;
33     }
34     auxiliar->proximo = novoNo;
35 }
36
37 void mostrarLista(Celula* sequencia) {
38     if (listaVazia(sequencia)) {
39         printf("[Lista Vazia]\n");
40         return;
41     }
42     Celula* auxiliar = sequencia;
43     while (auxiliar != NULL) {
44         printf("%d -> ", auxiliar->dado);
45         auxiliar = auxiliar->proximo;
46     }
47     printf("NULL\n");
48 }
49
50 void liberarLista(Celula** sequencia) {
51     Celula* auxiliar = *sequencia;
52     while (auxiliar != NULL) {
53         Celula* proxNo = auxiliar->proximo;
54         free(auxiliar);
55         auxiliar = proxNo;
56     }
57     *sequencia = NULL;
58 }
59
60 int maxLista(Node* lista) {
61     if (listaVazia(lista)) return -1;
62     int maiorValor = lista->dado;
63     Node* auxiliar = lista->proximo;
64     while (auxiliar != NULL) {
65         if (auxiliar->dado > maiorValor) {
66             maiorValor = auxiliar->dado;

```

```

67     }
68     auxiliar = auxiliar->proximo;
69 }
70 return maiorValor;
71 }
72
73 int minLista(Node* lista) {
74     if (listaVazia(lista)) return -1;
75     int menorValor = lista->dado;
76     Node* auxiliar = lista->proximo;
77     while (auxiliar != NULL) {
78         if (auxiliar->dado < menorValor) {
79             menorValor = auxiliar->dado;
80         }
81         auxiliar = auxiliar->proximo;
82     }
83     return menorValor;
84 }
85
86 float mediaLista(Node* lista) {
87     if (listaVazia(lista)) return 0.0;
88     int acumuladorSoma = 0;
89     int contadorItens = 0;
90     Node* auxiliar = lista;
91     while (auxiliar != NULL) {
92         acumuladorSoma += auxiliar->dado;
93         contadorItens++;
94         auxiliar = auxiliar->proximo;
95     }
96     return (float)acumuladorSoma / contadorItens;
97 }
98
99 int lenLista(Node* lista) {
100     int tamanho = 0;
101     Node* auxiliar = lista;
102     while (auxiliar != NULL) {
103         tamanho++;
104         auxiliar = auxiliar->proximo;
105     }
106     return tamanho;
107 }
108
109 Nodo* appendLista(Nodo* L1, Nodo* L2) {
110     if (listaVazia(L1)) return L2;
111     Nodo* auxiliar = L1;
112     while (auxiliar->proximo != NULL) {
113         auxiliar = auxiliar->proximo;
114     }
115     auxiliar->proximo = L2;
116     return L1;
117 }
118
119 Nodo* reverseLista(Nodo* lista) {
120     Nodo* anterior = NULL;
121     Nodo* atual = lista;
122     Nodo* proximoNo = NULL;
123     while (atual != NULL) {
124         proximoNo = atual->proximo;
125         atual->proximo = anterior;
126         anterior = atual;
127         atual = proximoNo;
128     }
129     return anterior;
130 }
131
132 int checarSequencia(Nodo* pPrincipal, Nodo* pSub) {

```

```
133     while (pSub != NULL) {
134         if (pPrincipal == NULL || pPrincipal->dado != pSub->dado) {
135             return 0;
136         }
137         pPrincipal = pPrincipal->proximo;
138         pSub = pSub->proximo;
139     }
140     return 1;
141 }
142
143 int isSubLista(Nodo* L1, Nodo* L2) {
144     if (listaVazia(L2)) return 1;
145     if (listaVazia(L1)) return 0;
146     Nodo* auxiliarP = L1;
147     while (auxiliarP != NULL) {
148         if (auxiliarP->dado == L2->dado) {
149             if (checarSequencia(auxiliarP, L2)) {
150                 return 1;
151             }
152         }
153         auxiliarP = auxiliarP->proximo;
154     }
155     return 0;
156 }
```

```
1  #ifndef LISTA_H
2  #define LISTA_H
3
4  typedef struct Celula {
5      int dado;
6      struct Celula* proximo;
7  } Celula;
8
9  typedef struct Celula Node;
10 typedef struct Celula Nodo;
11
12 Celula* criarLista();
13 int listaVazia(Celula* sequencia);
14 void inserirInicio(Celula** sequencia, int numero);
15 void inserirFim(Celula** sequencia, int numero);
16 void mostrarLista(Celula* sequencia);
17 void liberarLista(Celula** sequencia);
18
19 int maxLista(Node* lista);
20 int minLista(Node* lista);
21 float mediaLista(Node* lista);
22 int lenLista(Node* lista);
23 Nodo* appendLista(Nodo* L1, Nodo* L2);
24 Nodo* reverseLista(Nodo* lista);
25 int isSubLista(Nodo* L1, Nodo* L2);
26
27 #endif
```



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Pilha.h"
4
5  void criarPilha(EstruturaPilha* estrutura) {
6      estrutura->topoAtual = NULL;
7  }
8
9  int pilhaVazia(EstruturaPilha* estrutura) {
10     return (estrutura->topoAtual == NULL);
11 }
12
13 void empilhar(EstruturaPilha* estrutura, int numero) {
14     CelulaPilha* novoNo = (CelulaPilha*)malloc(sizeof(CelulaPilha));
15     if (novoNo == NULL) return;
16     novoNo->dado = numero;
17     novoNo->anterior = estrutura->topoAtual;
18     estrutura->topoAtual = novoNo;
19 }
20
21 int desempilhar(EstruturaPilha* estrutura) {
22     if (pilhaVazia(estrutura)) return -1;
23     CelulaPilha* auxiliar = estrutura->topoAtual;
24     int valor = auxiliar->dado;
25     estrutura->topoAtual = auxiliar->anterior;
26     free(auxiliar);
27     return valor;
28 }
29
30 void mostrarPilha(EstruturaPilha* estrutura) {
31     if (pilhaVazia(estrutura)) {
32         printf("[Pilha Vazia]\n");
33         return;
34     }
35     EstruturaPilha temp;
36     criarPilha(&temp);
37     while (!pilhaVazia(estrutura)) {
38         int v = desempilhar(estrutura);
39         printf("%d\n", v);
40         empilhar(&temp, v);
41     }
42     while (!pilhaVazia(&temp)) {
43         empilhar(estrutura, desempilhar(&temp));
44     }
45 }
46
47 void liberarPilha(EstruturaPilha* estrutura) {
48     while (!pilhaVazia(estrutura)) {
49         desempilhar(estrutura);
50     }
51 }
52
53 int buscarElemento(EstruturaPilha* estrutura, int numero) {
54     EstruturaPilha temp;
55     criarPilha(&temp);
56     int posicao = 1;
57     int encontrado = -1;
58     while (!pilhaVazia(estrutura)) {
59         int v = desempilhar(estrutura);
60         if (v == numero && encontrado == -1) {
61             encontrado = posicao;
62         }
63         empilhar(&temp, v);
64         posicao++;
65     }
66     while (!pilhaVazia(&temp)) {

```

```

67     empilhar(estrutura, desempilhar(&temp));
68 }
69 return encontrado;
70 }
71
72 int buscarERemover(EstruturaPilha* estrutura, int numero) {
73     EstruturaPilha temp;
74     criarPilha(&temp);
75     int encontrado = 0;
76     while (!pilhaVazia(estrutura)) {
77         int v = desempilhar(estrutura);
78         if (v == numero && !encontrado) {
79             encontrado = 1;
80         } else {
81             empilhar(&temp, v);
82         }
83     }
84     while (!pilhaVazia(&temp)) {
85         empilhar(estrutura, desempilhar(&temp));
86     }
87     return encontrado;
88 }
89
90 void removerPares(EstruturaPilha* estrutura) {
91     EstruturaPilha temp;
92     criarPilha(&temp);
93     while (!pilhaVazia(estrutura)) {
94         int v = desempilhar(estrutura);
95         if (v % 2 != 0) {
96             empilhar(&temp, v);
97         }
98     }
99     while (!pilhaVazia(&temp)) {
100         empilhar(estrutura, desempilhar(&temp));
101     }
102 }
103
104 void removerRepetidos(EstruturaPilha* estrutura) {
105     EstruturaPilha tempLimpa;
106     EstruturaPilha tempVerificadora;
107     criarPilha(&tempLimpa);
108     criarPilha(&tempVerificadora);
109     while (!pilhaVazia(estrutura)) {
110         int atual = desempilhar(estrutura);
111         int duplicado = 0;
112         while (!pilhaVazia(&tempLimpa)) {
113             int vVerifica = desempilhar(&tempLimpa);
114             if (vVerifica == atual) {
115                 duplicado = 1;
116             }
117             empilhar(&tempVerificadora, vVerifica);
118         }
119         while (!pilhaVazia(&tempVerificadora)) {
120             empilhar(&tempLimpa, desempilhar(&tempVerificadora));
121         }
122         if (!duplicado) {
123             empilhar(&tempLimpa, atual);
124         }
125     }
126     while (!pilhaVazia(&tempLimpa)) {
127         empilhar(estrutura, desempilhar(&tempLimpa));
128     }
129 }

```

```
1  #ifndef PILHA_H
2  #define PILHA_H
3
4  typedef struct CelulaPilha {
5      int dado;
6      struct CelulaPilha* anterior;
7  } CelulaPilha;
8
9  typedef struct {
10      CelulaPilha* topoAtual;
11  } EstruturaPilha;
12
13 void criarPilha(EstruturaPilha* estrutura);
14 int pilhaVazia(EstruturaPilha* estrutura);
15 void empilhar(EstruturaPilha* estrutura, int numero);
16 int desempilhar(EstruturaPilha* estrutura);
17 void mostrarPilha(EstruturaPilha* estrutura);
18 void liberarPilha(EstruturaPilha* estrutura);
19
20 int buscarElemento(EstruturaPilha* estrutura, int numero);
21 int buscarERemover(EstruturaPilha* estrutura, int numero);
22 void removerPares(EstruturaPilha* estrutura);
23 void removerRepetidos(EstruturaPilha* estrutura);
24
25 #endif
```